



AMERICAN  
UNIVERSITY  
OF BEIRUT



# Applied Cryptography

CMPS 297AD/396AI

Fall 2025

Part 1: Provable Security

## 1.6: Collision-Resistant Hash Functions

Nadim Kobeissi

<https://appliedcryptography.page>



Section 1

# Defining Hash Functions

# Symmetric primitive example: hash functions

## Reminder

### Hash Function Properties

- Takes input of **any size**
- Produces output of **fixed size**
- Is **deterministic** (same input → same output)
- Even a **tiny change** in input creates completely different output
- Is **efficient** to compute

SHA256(hello) =  
2cf24dba5fb0a30e26e83b2ac5  
b9e29e1b161e5c1fa7425e7304  
3362938b9824

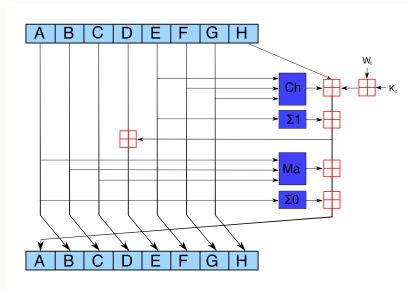
SHA256(hu!llo) =  
7835066a1457504217688c8f5d  
06909c6591e0ca78c254ccf174  
50d0d999cab0

**Note:** One character change →  
completely different hash!

# Expected properties of a hash function

## Reminder

- **Collision resistance:** computationally infeasible to find two different inputs producing the same hash.
- **Preimage resistance:** given the output of a hash function, it is computationally infeasible to reconstruct the original input.
- **Second preimage resistance:** given an input and an output, it's computationally infeasible to find another different input producing the same output.



SHA-2 compression function. Source: Wikipedia

# Hash functions: what are they good for?

## Reminder

- **Data integrity verification:** Hash a file. Later hash it again and compare hashes to check if the file has changed, suffered storage degradation, etc.
- **Proof of work:** Server asks client to hash something a lot of times before they can access some resource. Useful for anti-spam, Bitcoin mining, etc.

# Hash function collisions

## Hash Function Collision

A **collision** occurs when two different inputs produce the same hash output:  $x \neq y$  but  $H(x) = H(y)$

- Collisions **must exist** due to the pigeonhole principle: infinitely many possible inputs map to a finite set of outputs.
- A secure hash function makes finding collisions **computationally infeasible**.

## Collision Example:

Input 1: document.pdf  
SHA256 = a1b2c3...

Input 2: malware.exe  
SHA256 = a1b2c3...

**Problem:** Attacker could substitute malware.exe for document.pdf without detection!

# Hash function collisions

## Hash Function Collision

A **collision** occurs when two different inputs produce the same hash output:  $x \neq y$  but  $H(x) = H(y)$

- If an attacker can find collisions, they could substitute malicious data that produces the same hash as legitimate data.
- The **birthday paradox** shows collisions are found faster than expected (after  $\approx 2^{n/2}$  attempts for an  $n$ -bit hash).

## Collision Example:

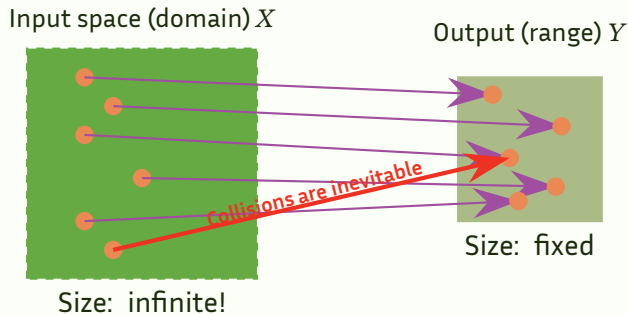
Input 1: document.pdf  
SHA256 = a1b2c3 ...

Input 2: malware.exe  
SHA256 = a1b2c3 ...

**Problem:** Attacker could substitute malware.exe for document.pdf without detection!

$$\text{PRF} : F_k = X \rightarrow Y$$

- We want the mapping to be:
  - One-way
  - "Randomized"
  - Relations between inputs not reflected in outputs





# Birthday paradox

- In a classroom of 50 students, what's the probability that at least two share a birthday?
  - 2%?
  - 20%?
  - 50%?
  - 97%?
- Many people guess around 15-20%, but the actual probability is about 97%!
- This is counterintuitive because **we're not looking for a specific birthday - we're looking for *any* match among all possible pairs.**
- With 50 students, we have  $\binom{50}{2} = 1,225$  possible pairs to check!<sup>a</sup>

---

<sup>a</sup> $\binom{50}{2}$  is a binomial coefficient. It means: "how many ways can I choose two different items from a set of 50?"

# Birthday paradox

- With just 23 people, probability exceeds 50%
- Formula for  $n$  people:

$$P = 1 - \frac{365!}{(365 - n)! \cdot 365^n}$$

- Implication: Finding collisions in a space of size  $N$  happens with roughly  $\sqrt{N}$  samples.
- **This is why many cryptographic systems need large output spaces!**

# Birthday probabilities

- Many cryptographic algorithms fail if two executions sample the same random value.
- General question: If we take  $q$  independent uniform samples from a set of  $N$  items, what's the probability some value is chosen more than once?
- This probability is called  $\text{BIRTHDAY}(q, N)$

$$\text{BIRTHDAY}(q, N) = 1 - \prod_{i=1}^{q-1} \left(1 - \frac{i}{N}\right)$$

- Surprisingly high probability!
- With  $q \approx 1.2\sqrt{N}$ , probability  $\approx 0.5$
- For birthdays: only need 23 people for  $> 50\%$  chance

## Section 2

# Attacks on Hash Functions

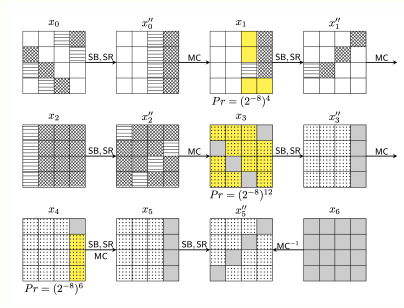
# Example hash functions

## From historical to modern

- **Historical/broken hash functions:**
  - **MD4** (1990): 128-bit, completely broken, collisions found instantly
  - **MD5** (1992): 128-bit, collisions found in 2004, still widely misused
  - **SHA-0** (1993): 160-bit, withdrawn shortly after release
  - **SHA-1** (1995): 160-bit, practical collisions demonstrated in 2017
- **Current standard hash functions:**
  - **SHA-2 family** (2001): SHA-224, SHA-256, SHA-384, SHA-512
  - **SHA-3 family** (2015): Based on the Keccak sponge construction
- **Modern alternatives:**
  - **BLAKE2** (2012): Faster than MD5, more secure than SHA-3
  - **BLAKE3** (2020): Highly parallelizable, extremely fast

# Evaluating a hash function's quality

- Recall:
  - Asymmetric primitives** are based on mathematical problems, can be mathematically proven secure (given assumptions!)
  - Symmetric primitives** (encryption, hashing...) are statistically, empirically, heuristically shown to be secure, not proven secure.
  - The more cryptanalysis they survive, the higher confidence we have in their security.



Cryptanalysis of AES.

# What about attacks on hash functions?

- Despite their security properties, hash functions can be vulnerable.
- We'll explore key attack strategies:
  - Precomputation attacks using rainbow tables.
  - Birthday attacks exploiting the birthday paradox.
  - Ways attackers try to find collisions.
- Then we'll discuss countermeasures like salting.
- We'll then show additional attacks on hash function, and discuss their practical evolution.

# Precomputation attacks

## Why salts matter

- Consider a server storing hashes of users' passwords.
- With **unsalted** hashes, an attacker can:
  - Pre-compute hashes of common passwords.
  - Create a dictionary mapping  $H(\text{password}) \rightarrow \text{password}$ .
  - Instantly crack common passwords after a breach.
- All hard work is done **before** the attack.
- Results can be **reused** against any system using the same hash function.



# Rainbow tables

## Rainbow Tables

**Rainbow tables** are specialized data structures that store precomputed hash values to efficiently crack passwords with minimal computation during the actual attack.

- Represent a time-memory tradeoff for password cracking.
- Store chains of hashes rather than individual hash-password pairs.
- Can crack any password within the table's character set and length parameters.
- Completely defeated by properly salted hashes.

# Salt in hash functions

## Preventing precomputation attacks

### Hash Salt

A **salt** is a random value that is concatenated with the input before hashing:  $H(S\|X)$  where  $S$  is the salt and  $X$  is the original input.

- The salt doesn't need to be secret, but should be:
  - Random and unique for each input.
  - Long enough to prevent precomputation ( $\geq 16$  bytes).
  - Stored alongside the hash value.
- Salting prevents attackers from using precomputed tables (rainbow tables) to find hash values.
- Without salt, same inputs always hash to same outputs, enabling reusable attacks.

# Practical applications of salting

## Real-world examples

- **Web application user accounts**
  - Different users with same password have different hashes.
  - Breach of one site doesn't compromise accounts on others.
- **Password reset tokens**
  - Salt + timestamp prevents token reuse.
  - Each reset request generates unique token.
- **API key generation**
  - Salting ensures unique keys even for similar services.
  - Allows key rotation without pattern recognition.
- **Database record anonymization**
  - Different salts for different datasets prevent correlation attacks.

# Hash functions in the real world

- Hash functions you encounter “in the wild” are **standardized** as **unsalted** functions.
- They have only a single input argument: a string to be hashed.

# How salt neutralizes precomputation

- With salted hashes, precomputation becomes useless:
  - Attacker doesn't know the salts until they've already breached the system.
  - Can't precompute  $H(S\|X)$  without knowing  $S$ .
- Using **different** salt for each user:
  - Forces attacker to choose which user to attack.
  - Attacking two victims requires twice the effort.
  - No more "attack all users for the price of one".
- Even if attacker obtains both hash and salt, they still need to brute-force each password individually.

# Length extension attacks

## SHA-1 and SHA-2 vulnerability

### Length Extension Attack

A **length extension attack** allows an attacker who knows  $H(M)$  to calculate  $H(M\|P)$  for some padding  $P$  without knowing  $M$  itself.

- Affects hash functions built using the Merkle-Damgård construction:
  - SHA-1, SHA-256, SHA-512, MD5, and others
  - Not SHA-3 (which uses the sponge construction)
- An attacker who knows  $H(\text{message})$  and the length of message can compute  $H(\text{message}\|\text{padding}\|\text{extra\_data})$ .
- This can break naive authentication schemes using  $H(\text{secret}\|\text{message})$ .

# Length extension attacks

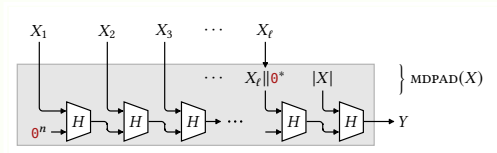
## Real-world implications

- Consider a web API using:
  - URL: `api.com/data?val=foo&hash=123abc`
  - Hash calculated as:  $H(\text{secret} \parallel \text{val}=\text{foo})$
  - Server verifies request by checking if hash matches
- An attacker can:
  - Take a valid hash value
  - Compute a new hash for `val=foo&malicious=true`
  - Without knowing the secret!

# The Merkle-Damgård construction

## SHA-1, SHA-2

- Key components:
  - Message is padded to a multiple of block size.
  - Split into fixed-size blocks.
  - Processes blocks sequentially through a compression function.
  - Each step combines previous state with current block.<sup>a</sup>
- If compression function is collision-resistant, so is the hash function.



Source: The Joy of Cryptography

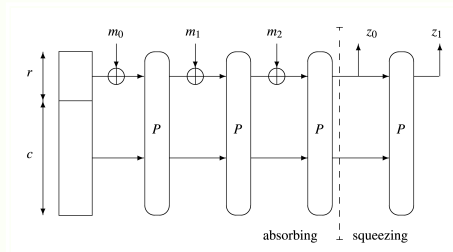
<sup>a</sup>Interactive visualization: <https://sha256algorithm.com>



# The Sponge construction

## SHA-3, BLAKE

- Two phases:
  - **Absorbing:** Input blocks are XORed into the state and processed.
  - **Squeezing:** Output blocks are extracted from the state.
- Naturally resistant to length extension attacks.
- Versatile: Can produce output of any desired length.



Source: The Hash Function BLAKE (by Jean-Philippe Aumasson)

# How the sponge construction prevents length extension

- In Merkle-Damgård, the final state becomes the hash output directly.
- This allows attackers to continue from that state without knowing the input.
- The sponge construction protects against this by:
  - Maintaining a larger internal state than the output size.
  - Only revealing part of the state as output.
  - The hidden portion (capacity) remains secret.

## Sponge State:

Rate ( $r$ ): 1088 bits  
[Revealed in output]

Capacity ( $c$ ): 512 bits  
[Hidden from attacker]

**Key:** Without knowing the full state, attacker cannot continue hashing!

# HMAC: Protection against length extension

## HMAC

**HMAC** (Hash-based Message Authentication Code) is a specific construction for calculating a message authentication code (MAC) using a cryptographic hash function and a secret key.

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| m))$$

- *opad* and *ipad* are fixed padding values.
- This nested construction defeats length extension attacks.
- Widely used in TLS, IPsec, and other security protocols.
- Provides both integrity and authentication.

# Xiaoyun Wang's MD5 Collision

- In 2004, Chinese researcher Xiaoyun Wang demonstrated the first practical MD5 collision.
- Presented at the CRYPTO 2004 rump session, causing a sensation in the cryptographic community.
- Interesting backstory: Wang's team initially got confused by endianness issues.
- The Chinese translation of Bruce Schneier's "Applied Cryptography" had incorrectly translated the MD5 algorithm.
- The translation got the byte ordering (endianness) wrong, making their early collision attempts fail.
- Once corrected, they succeeded in breaking MD5 much faster than expected.
- This discovery accelerated the migration away from MD5 in security applications.

# MD5 Chosen-Prefix Collision

## Attacking certificate authorities

- In 2008, researchers created a rogue CA certificate using MD5 collisions.
- Exploited weaknesses in how certificates were issued by a legitimate CA.
- By predicting serial numbers and timestamps, they crafted a malicious certificate with the same MD5 hash.
- Created a fake Certificate Authority that browsers would trust!
- Demonstrated at the 25th Chaos Communication Congress.
- Led to MD5 being rapidly phased out for digital signatures.

# SHA-1 Collision Attack (SHAttered)

## The first practical SHA-1 collision

- In 2017, Google and CWI Amsterdam researchers demonstrated the first practical collision for SHA-1.<sup>a</sup>
- Created two different PDF files with identical SHA-1 hashes.
- Required about 6,500 CPU years and 110 GPU years of computation.
- Cost estimate: approximately \$110,000 using cloud computing.
- Proved that SHA-1 should no longer be used for security-critical applications.

---

<sup>a</sup><https://appliedcryptography.page/paper/#shattered-sha1>

Section 3

## Password Hashing

# Password-based authentication

## Common use of hash functions

- Passwords are the most common authentication mechanism.
- Server must verify if a user's password is correct.
- Different methods for password storage:
  - Storing in the clear (extremely bad)
  - Encryption (also bad)
  - Unsalted hashing (better, but problematic)
  - Salted hashing (good practice)
  - Specialized password hashing (best practice)
- We compare methods by asking: what happens if the server is compromised?



# Storing passwords in the clear

## The worst approach

- Server simply stores exact passwords
- Password verification is trivial:
  - User enters password  $P'$
  - Server checks if  $P' == P$
- If server is compromised, attacker immediately learns all passwords.
- Users often reuse passwords across sites, compounding the damage.

### Database Storage:

```
username:  alice
password:  secret123

username:  bob
password:  password1
```

**Problem:** Attacker who accesses the database instantly has everyone's actual passwords!

# Encrypting passwords

## A common mistake

- Some systems try to encrypt passwords:
  - Store  $\text{Enc}(K, P)$  instead of  $P$ .
  - Decrypt when verifying login.
- Problem: The decryption key  $K$  must be accessible to the server.
- If an attacker compromises the server, they get both:
  - The encrypted passwords.
  - The decryption key.
- Result: Encryption adds no security!

### Encrypted Password Storage:

username: alice

password: A7F3D92C...

decryption\_key: 5B8E1...

**Problem:** Server needs the key to verify passwords, so attacker gets the key too!

# Unsalted password hashing

## A basic approach

- For a user with password  $P$ , the server stores  $h = H(P)$ .
- On login, server checks if  $H(P') == h$ .
- Advantages over cleartext:
  - Attacker doesn't immediately get passwords.
  - Must spend effort guessing and hashing.
- Problems:
  - Vulnerable to precomputation (rainbow tables).
  - Same password = same hash for all users.

### Unsalted Hash Storage:

username: alice  
hash: 5f4dcc3b5aa...

username: bob  
hash: 5f4dcc3b5aa...

**Problem:** Alice and Bob have the same password (visible from identical hashes)!

# Limitations of unsalted hashing

- Attacker's work can be **precomputed**:
  - Generate lookup tables before compromising the server.
  - Rainbow tables provide time-memory tradeoff.
- Same effort cracks multiple accounts:
  - If users have same password, they have same hash.
  - Cracking 1,000 accounts may be no harder than cracking one.
- Common passwords are instantly recognized:
  - $H(\text{password123})$  is well-known.
  - Attacker doesn't even need to compute hashes for common passwords.

# Salted password hashing

## The proper approach

### Salted Hash Authentication

For a user with password  $P$ , the server stores:

- A random salt  $S$  (unique per user)
- The hash value  $h = H(S\|P)$

To verify a login with password  $P'$ , the server checks if  $H(S\|P') == h$ .

### Salted Hash Storage:

```
username: alice  
salt: 8a7b3c...  
hash: e8f74d...
```

```
username: bob  
salt: 2c9d5e...  
hash: 9f82e1...
```

**Note:** Even if Alice and Bob have the same password, they'll have different hashes!

# Benefits of salted hashing

- Prevents precomputation attacks:
  - Rainbow tables become useless.
  - Attacker must compute hashes after server compromise.
- Requires unique effort per user:
  - Each password must be attacked individually.
  - Cracking 1,000 accounts requires 1,000 times the effort.
- Same passwords produce different hashes:
  - Identical passwords across users aren't visible.
  - Doesn't leak information about password reuse.
- **Always salt your password hashes!**

# Specialized password hashing functions

## Beyond regular hash functions

- Regular cryptographic hash functions (SHA-2, SHA-3) are designed to be **fast**.
- But password hashing, slower is better!
  - Makes attacks more expensive.
  - Users don't notice milliseconds of delay.
  - Attackers trying billions of guesses definitely notice.
- Simple approach: Iterate the hash function thousands of times.
  - $H(S, H(S, \dots H(S, P) \dots))$  with many iterations.
  - **Obsolete examples:** PBKDF2.
  - **Modern examples:** Argon2, Scrypt

# PBKDF2: Password-Based Key Derivation Function 2

## PBKDF2

**PBKDF2** (Password-Based Key Derivation Function 2) is a key derivation function that applies a pseudo-random function (like HMAC) to the input password along with a salt to produce a derived key.

- Key components:
  - **Password**: Secret input.
  - **Salt**: Provides randomization.
  - **Iteration count**: Controls computational cost.
  - **PRF**: Usually HMAC-SHA1 or HMAC-SHA256.

$$U_1 = \text{PRF}(\text{Password}, \text{Salt} || \text{Int}(1))$$

$$U_2 = \text{PRF}(\text{Password}, U_1)$$

$$U_3 = \text{PRF}(\text{Password}, U_2)$$

$$\vdots$$

$$U_c = \text{PRF}(\text{Password}, U_{c-1})$$

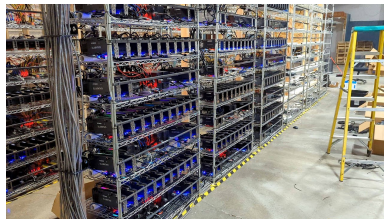
$$T_1 = U_1 \oplus U_2 \oplus \dots \oplus U_c$$



# Why not PBKDF2?

## Vulnerable to specialized hardware

- **Problem:** Attackers can build specialized hardware.
  - ASICs can compute hashes millions of times faster.
  - Iteration alone isn't enough protection.
- **Solution:** Memory-hard functions
  - Require significant amount of memory.
  - Hardware specialization offers less advantage.
  - Both attackers and defenders use similar memory tech.



A GPU farm, likely built to mine Bitcoin, but which can also be used to do a lot of hashing really fast (Bitcoin mining is just calculating a lot of SHA-256 hashes!)

# Blockchain and hash functions

- Bitcoin and blockchain technology rely heavily on hash functions:
  - **Proof-of-work:** Finding partial hash collisions (leading zeros).
  - **Merkle trees:** Efficient verification of large datasets.
  - **Block chaining:** Each block contains the hash of the previous block.
- Miners try to find inputs that produce hashes with specific properties.
- The extreme computational cost of Bitcoin mining ( $\approx 90$  TWh/year) demonstrates hash function security.
- Bitcoin mining consumes more electricity than many countries.

# Scrypt: A memory-hard password hashing function

## Scrypt

**Scrypt** is a password-based key derivation function designed to be computationally and memory-intensive, making it resistant to hardware acceleration attacks.

- Created by Colin Percival in 2009 specifically to make attacks costly.
- Key components:
  - **PBKDF2**: Initial and final mixing of password and salt.
  - **ROMix**: Core memory-hard function.
  - **BlockMix**: Based on Salsa20/8 core function.
- Parameters:
  - N: CPU/memory cost
  - r: Block size
  - p: Parallelization

# Why Scrypt is memory-hard

- Scrypt enforces memory hardness through its ROMix function:
  - Creates a large array of pseudorandom values (often many MB).
  - Each new value depends on previous values in unpredictable patterns.
  - Forces the entire array to be stored in memory during computation.
- Memory hardness creates multiple benefits:
  - GPUs/ASICs have limited memory per processing unit.
  - Memory is expensive and scales poorly on specialized hardware.
  - Time-memory tradeoffs are possible but highly unfavorable.
  - Using less memory makes computation exponentially slower.
- Even with custom hardware, attackers face similar costs to defenders.
- Proven to be maximally memory-hard!<sup>a</sup>

---

<sup>a</sup><https://appliedcryptography.page/paper/#scrypt-memory>

# Password cracking speeds

## The importance of slow hashing

- Modern hardware can compute hash functions at astonishing speeds.
- A single high-end GPU can compute:
  - MD5: 25+ billion hashes/second
  - SHA-1: 10+ billion hashes/second
  - SHA-256: 2+ billion hashes/second
  - bcrypt (cost 12): 20,000 hashes/second
  - Scrypt: <1,000 hashes/second
- This is why specialized password hashing algorithms are essential.
- The stark speed difference (billions vs. thousands) is a critical security factor.

# Insights from Hashcat 7.0.0

## The evolving landscape of password cracking

### Hashcat

**Hashcat** is the world's fastest and most advanced password recovery tool, supporting over 300 hash algorithms and multiple attack modes.

- Released after 2+ years of development with 900,000+ lines of code changed.<sup>a</sup>
- 105 contributors show the scale of the password cracking community.
- Major architectural changes reveal trends in both attack and defense strategies.

---

<sup>a</sup><https://hashcat.net/forum/thread-13330.html>

# Trend 1: Memory-hard functions are winning

But attackers are adapting

- **Argon2 support added** - defenders are adopting memory-hard functions.
- **Scrypt optimization**: +320% performance improvement shows attackers aren't giving up.
- **Memory management rewrite**: Removed 4GB allocation caps, enabling full device memory usage.
- This arms race validates our discussion about memory-hard functions!

## Performance Gains:

Scrypt: +320%

RAR3: +54%

NetNTLMv2: +223%

**Note:** Even memory-hard functions face continuous optimization pressure!

## Trend 2: Hardware diversity in attacks

- **Assimilation Bridge:** Integrate CPUs, FPGAs, and embedded devices.
- **Multiple backend support:**
  - HIP (AMD GPUs)
  - Metal (Apple Silicon)
  - Virtual Backend Devices
- **Python Bridge Plugin:** Rapid prototyping of new attack methods.
- Attackers now leverage every available compute resource, not just traditional GPUs.

### Attack Surface Expansion:

Traditional: GPU farms



Modern: Heterogeneous computing

**Implication:** Defenders must assume attackers have access to diverse, optimized hardware ecosystems.



# Trend 3: Real-world targets dominate

- **58 new application-specific hashes:**
  - MetaMask (cryptocurrency wallets)
  - Microsoft Online Accounts
  - LUKS2 (disk encryption)
  - GPG, OpenSSH (security tools)
- **20 new extraction tools** for BitLocker, APFS, VirtualBox.
- **Hash-mode autodetection:** Attackers don't even need to know the algorithm!
- Focus has shifted from breaking algorithms to extracting and cracking real-world implementations.

## Attack Evolution:

- Past: Break MD5/SHA-1
- Today: Extract from:
  - Encrypted disks
  - Password managers
  - Cryptocurrency wallets
  - Cloud accounts

**Lesson:** Security depends on implementation, not just algorithm choice!

# Best practices for password hashing

- Use a dedicated password hashing function (not raw SHA-256).
- Always use random, unique salts (at least 16 bytes).
- Store salts alongside hashes - they don't need to be secret.
- Consider modern memory-hard functions like Scrypt.
  - Tune parameters based on your server capabilities.
  - Aim for 0.5-1 second processing time per hash.
- Keep your hashing mechanism current:
  - Cryptographic advice changes as attacks improve.
  - Be prepared to migrate to stronger algorithms.

# “Migrate to stronger algorithms?”

- Cryptographic algorithms have limited lifespans:
  - New attacks emerge over time.
  - Computing power continuously increases.
  - Quantum computers threaten some algorithms.
- **Protocol agility** refers to a system's ability to:
  - Support multiple cryptographic algorithms.
  - Negotiate which algorithm to use.
  - Transition smoothly as algorithms are deprecated.
- Challenges with protocol agility:
  - Complexity leads to implementation errors.
  - Downgrade attacks target weakest supported option.
  - Backward compatibility vs. security tradeoffs.
- We'll talk a lot more about all of this in Part 2 of the course.

Section 4

## The Random Oracle Model

# Hash functions can't be truly pseudorandom

- Any deterministic algorithm cannot produce truly random output.
- Hash functions only statistically approximate pseudorandomness:
  - They attempt to distribute outputs uniformly across their range,
  - They aim to make outputs appear uncorrelated with inputs,
  - But they remain deterministic functions.
- But we analyze hash functions as if they were truly random oracles!
  - This creates a gap between theory and implementation.
  - Implementations can only ever approximate ideal properties.

# The Random Oracle Model

## Random Oracle Model

The **Random Oracle Model** is a theoretical framework that models hash functions as truly random functions that can be queried by all parties.

- A random oracle is an idealized black box that:
  - Responds to new queries with truly random outputs.
  - Consistently returns the same output when queried with the same input.
  - Can be accessed by all parties (both honest and adversarial).
- Provides a mathematical abstraction that's easier to analyze than actual hash functions

# Using the Random Oracle Model

- Used to prove security of cryptographic protocols:
  - Allows rigorous mathematical analysis.
  - Simplifies proofs that would otherwise be intractable.
  - Many important schemes are proven secure in this model.
- Example: OAEP for RSA encryption.
- Provides confidence in schemes before deployment.
- **Proving in the ROM:**
  1. Replace hash function with random oracle
  2. Show adversary has negligible advantage.
  3. Conclude the scheme is secure (assuming the hash function behaves like a random oracle).

# Limitations of the Random Oracle Model

- **Gap between theory and practice:**
  - No real hash function can perfectly implement a random oracle.
  - Schemes secure in the RO model might be insecure when implemented with real hash functions.
- **Artificial counterexamples exist:**
  - Researchers have constructed schemes that are:
  - Provably secure in the RO model, but,
  - Provably insecure with any real hash function.<sup>a</sup>

---

<sup>a</sup><https://appliedcryptography.page/paper/#rom-methodology>





AMERICAN  
UNIVERSITY  
OF BEIRUT



# Applied Cryptography

CMPS 297AD/396AI

Fall 2025

Part 1: Provable Security

## 1.6: Collision-Resistant Hash Functions

Nadim Kobeissi

<https://appliedcryptography.page>

